

Universidad Nacional de Colombia

Matemáticas Discretas II

Proyecto MD

Demostración de Joyal a la Fórmula de Cayley

Documentación técnica del repositorio

Profesor: Arles Ernesto Rodríguez Portela

Fecha: Julio de 2026

Estudiantes: Martín Lora Caro

Cristian Andrés Díaz Ortega

Jhon Edison Prieto Artunduaga

Johan Alexander Peña Ortiz

Bogotá, Colombia

Índice

1. Proyecto general	3
1.1. Propósito de la aplicación	3
1.2. Objetivos principales	3
1.3. Motivación	3
2. Arquitectura del sistema	3
2.1. Vista general	3
2.2. Organización de carpetas	4
2.3. Estructura del repositorio	4
3. Tecnologías utilizadas	5
4. Algoritmos y estructuras de datos	5
4.1. Union-Find (Disjoint Set)	5
4.2. Búsqueda en profundidad	6
4.3. Búsqueda en anchura	6
4.4. Detección de ciclos en funciones	6
4.5. Algoritmo de Bareiss	7
4.6. Inversa modular y aritmética modular	7
4.7. Construcción de la matriz de cifrado	7
4.8. Cifrado Hill	7
5. Conceptos de teoría de grafos	8
5.1. Funciones	8
5.2. Árboles	8
5.3. Grafos dirigidos	8
5.4. Ciclos	8

5.5. Self-loops	8
5.6. Representación de vértices y aristas	8
6. Flujo de la aplicación	9
6.1. Construcción de función desde árbol	9
6.2. Construcción de árbol desde función	9
6.3. Generación aleatoria de funciones	9
6.4. Encriptación y desencriptación	9
6.5. Renderizado de grafos	10
6.6. Renderizado de loops	10
7. Detalles de implementación	10
7.1. Clases principales	10
7.2. Decisiones de diseño	10
7.3. Relación entre componentes	11
8. Capturas y figuras	11
9. Posibles mejoras futuras	13
10. Conclusiones	13
A. Fragmento representativo de código	13

1 Proyecto general

1.1 Propósito de la aplicación

El repositorio contiene una aplicación de escritorio desarrollada con Python y CustomTkinter para explorar una demostración combinatoria de la fórmula de Cayley mediante la biyección de Joyal entre árboles etiquetados y funciones. La aplicación también integra un esquema de cifrado Hill sobre un alfabeto extendido de 30 símbolos.

1.2 Objetivos principales

- Construir árboles a partir de funciones y funciones a partir de árboles.
- Visualizar grafos, vértebras, aristas dirigidas y bucles de autoreferencia.
- Generar claves matriciales a partir de funciones para encriptación y desencriptación.
- Mantener una interfaz gráfica interactiva, clara y consistente.

1.3 Motivación

El proyecto combina teoría de grafos, combinatoria, aritmética modular y visualización interactiva para ofrecer una herramienta educativa que conecta conceptos abstractos con una interfaz manipulable por el usuario.

2 Arquitectura del sistema

2.1 Vista general

La arquitectura separa la interfaz gráfica de la lógica de negocio. La carpeta `gui/` contiene las vistas y el canvas de dibujo; la carpeta `logic/` contiene las rutinas matemáticas, de grafos y de criptografía.

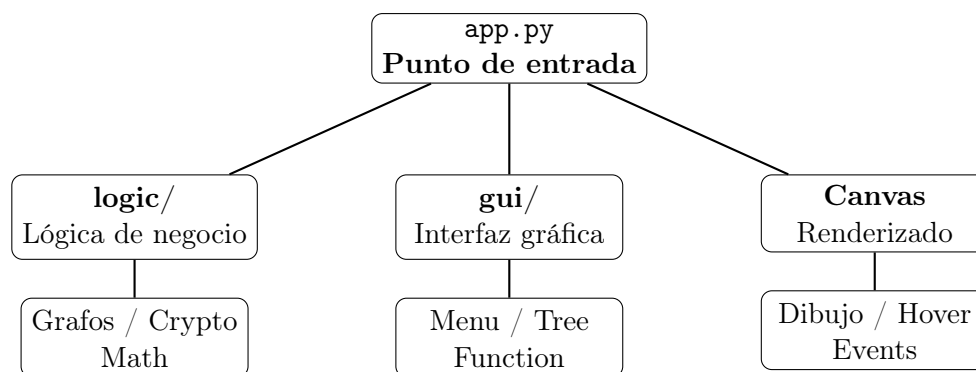


Figura 1: Arquitectura de alto nivel del repositorio.

2.2 Organización de carpetas

Carpeta o archivo	Responsabilidad
app.py	Inicializa la aplicación y maneja errores de ejecución.
gui/	Contiene la interfaz basada en CustomTkinter.
logic/	Implementa algoritmos y estructuras de datos.
README.md	Documentación funcional y de instalación.
run.bat	Instalación y arranque en Windows.
requirements.txt	Dependencias de Python.

Tabla 1: Estructura funcional del repositorio.

2.3 Estructura del repositorio

El siguiente listado muestra la estructura completa de archivos y carpetas del repositorio:

Listing 1: Estructura de archivos del repositorio CayleyTree

```

CayleyTree/
|-- app.py                <- PUNTO DE ENTRADA (ejecutar este
    archivo)
|-- app_original.py       <- Codigo original (respaldo)
|-- run.bat               <- EJECUTABLE WINDOWS (doble click
    aqui!)
|
|-- README.md             <- Documentacion funcional e
    instalacion
|-- requirements.txt       <- Dependencias de Python
|
|-- logic/                <- LOGICA DE NEGOCIO
|   |-- __init__.py

```

```

|  |-- graph_logic.py          <- Grafos , arboles , Union-Find , DFS
|  |-- crypto_logic.py        <- Cifrado Hill Cipher 9x9
|  '-- math_utils.py          <- Matrices , determinantes , inversa
    modular
|
|-- gui/                      <- INTERFAZ GRAFICA
    |-- __init__.py
        |-- main_window.py      <- Ventana principal y menu
            |-- tree_view.py      <- Vista Arbol -> Funcion
                |-- function_view.py <- Vista Funcion -> Arbol
                    '-- graph_canvas.py <- Canvas para
                        dibujar grafos

```

3 Tecnologías utilizadas

Categoría	Tecnología	Uso
Lenguaje	Python	Implementación general.
Interfaz	Tkinter / CustomTkinter	Ventanas, controles y canvas.
Cálculo numérico	NumPy	Operaciones matriciales.
Distribución	Batch script (run.bat)	Instalación y ejecución en Windows.
Formato documental	LaTeX	Reporte técnico.
Visualización de grafos	Canvas personalizado	Dibujo manual de nodos y aristas.

Tabla 2: Tecnologías empleadas en el proyecto.

No se utiliza una biblioteca externa de grafos como Graphviz o NetworkX para el dibujo. La visualización se implementa manualmente sobre `tkinter.Canvas`, lo que permite controlar la estética, la geometría de aristas, las vértebras y los bucles.

4 Algoritmos y estructuras de datos

4.1 Union-Find (Disjoint Set)

La clase `GraphLogic` utiliza conjuntos disjuntos para detectar ciclos al agregar aristas. En `agregar_arista` se verifica si dos vértices pertenecen al mismo conjunto antes de unirlos.

- Estructura: arreglo `parent`.
- Operaciones: `find` con compresión de caminos y `union`.
- Complejidad: casi constante amortizada, $O(\alpha(n))$.

Listing 2: Esquema de validación de aristas.

```
si find(v1) == find(v2):  
    rechazar arista  
si no:  
    agregar arista  
    union(v1, v2)
```

4.2 Búsqueda en profundidad

El método `depthfirstsearch` encuentra el camino entre vértices inicial y final dentro del árbol. Se usa para construir la vértebra en el modo árbol $\rightarrow$ función.

- Estructura auxiliar: conjunto `visited` y lista `path`.
- Complejidad: $O(V + E)$.

4.3 Búsqueda en anchura

El método `dirigir_vertices` calcula distancias al vértice final con una cola `deque`. Con esas distancias orienta las aristas que no pertenecen a la vértebra.

- Estructura: `collections.deque`.
- Propósito: orientar aristas hacia el vértice final.
- Complejidad: $O(V + E)$.

4.4 Detección de ciclos en funciones

El método `vertices_en_ciclo` construye un diccionario de adyacencias a partir de la función y busca caminos alternativos al retirar temporalmente una arista. El objetivo es identificar los vértices que forman ciclos y reconstruir la vértebra en el modo función $\rightarrow$ árbol.

4.5 Algoritmo de Bareiss

En `logic/math_utils.py` se implementa `determinante_bareiss`, una versión exacta del algoritmo de Bareiss para determinantes enteros. Esta elección evita errores de redondeo propios de métodos numéricos aproximados.

- Entrada: matriz cuadrada entera.
- Salida: determinante exacto.
- Complejidad: $O(n^3)$.

4.6 Inversa modular y aritmética modular

La función `inverse_matrix_mod` calcula la inversa módulo 30 cuando el determinante es invertible. Se usa el algoritmo extendido de Euclides para obtener el inverso modular.

- `egcd`: obtiene máximo común divisor e inversos.
- `modinv`: calcula inversos modulares.
- `adjugate_matrix`: construye la adjunta.

4.7 Construcción de la matriz de cifrado

La función `get_matrix_from_function` genera una matriz $n \times n$ a partir de la función. Si la matriz generada no es invertible módulo 30, se devuelve una matriz diagonal ajustada con entradas coprimas con 30.

$$M[i][j] = (f[i] \cdot (j + 1) + (f[j] + 1)) \text{ mód } 30 \quad (1)$$

4.8 Cifrado Hill

`CryptoEngine` utiliza la matriz anterior para cifrar y descifrar por bloques. El texto se codifica con longitud embebida, se convierte a números, se multiplica por la clave y luego se decodifica.

- Alfabeto extendido: A-Z, Ñ, coma, punto y espacio.
- Bloque: tamaño n , normalmente igual al número de vértices.
- Complejidad: dominada por multiplicación matricial por bloques.

5 Conceptos de teoría de grafos

5.1 Funciones

En el proyecto, una función es una lista de enteros donde cada posición representa un vértice y el valor almacenado es el destino de la aplicación. La representación interna usa índices base 0, aunque la interfaz muestra etiquetas base 1.

5.2 Árboles

Un árbol se modela como un grafo conexo sin ciclos. En el modo árbol $\rightarrow$ función el usuario construye manualmente un árbol válido; en el modo función $\rightarrow$ árbol el grafo funcional se transforma en la estructura arbórea equivalente.

5.3 Grafos dirigidos

Las funciones se representan como grafos dirigidos. Cada vértice tiene una arista de salida. En el canvas, las aristas dirigidas se dibujan con flecha y color púrpura; la vértebra se resalta con línea roja punteada.

5.4 Ciclos

Los ciclos aparecen naturalmente en grafos funcionales. El proyecto detecta los vértices del ciclo y los reorganiza para reconstruir la vértebra.

5.5 Self-loops

Un self-loop aparece cuando un vértice apunta a sí mismo. La visualización se renderiza como un lazo suave externo con arrowhead, integrándose con el estilo general del canvas.

5.6 Representación de vértices y aristas

Los vértices se dibujan como círculos con etiqueta centrada. Las aristas simples usan líneas grises, las dirigidas usan flechas púrpura y las vértebras usan trazos rojos punteados. Esta separación visual facilita distinguir estructura, dirección y segmentos especiales.

6 Flujo de la aplicación

6.1 Construcción de función desde árbol

1. El usuario conecta vértices sin formar ciclos.
2. `GraphLogic.agregar_arista` valida cada conexión.
3. Se eligen vértices inicial y final.
4. `construir_funcion_desde_arbol` obtiene la vértebra.
5. Se asignan destinos a los vértices de la vértebra.
6. Se orientan las aristas restantes hacia el vértice final.
7. `CryptoEngine.set_key_from_function` genera la clave de cifrado.

6.2 Construcción de árbol desde función

1. El usuario ingresa o genera una función aleatoria.
2. `construir_arbol_desde_funcion` detecta ciclos.
3. Se extrae la vértebra y se separan aristas dirigidas.
4. La vista permite mostrar el resultado como árbol con vértice o como árbol simple.

6.3 Generación aleatoria de funciones

La función auxiliar `generate_random_function` crea una función de tamaño n con valores uniformes entre 1 y n . Se usa tanto en `FunctionView` como en `CryptoView` para compartir comportamiento.

6.4 Encriptación y desencriptación

1. El usuario define una función.
2. La función se convierte en matriz de clave.
3. El texto se normaliza y se codifica a números.
4. Se aplica Hill Cipher módulo 30.
5. Se reconstruye el texto cifrado o descifrado.

6.5 Renderizado de grafos

La clase `GraphCanvas` concentra el dibujo. Entre sus responsabilidades están:

- calcular posiciones de vértices en círculo,
- detectar hover del mouse,
- dibujar aristas simples y dirigidas,
- dibujar vértebras,
- dibujar self-loops como curvas suaves.

6.6 Renderizado de loops

El loop se dibuja como una spline externa que sale del vértice hacia el exterior del grafo, recorre una curva suave y regresa cerca del nodo con una flecha consistente con las aristas dirigidas. La implementación evita depender de un símbolo abstracto y busca una lectura visual más cercana a la convención habitual de visualización de grafos.

7 Detalles de implementación

7.1 Clases principales

Clase	Responsabilidad
<code>MainWindow</code>	Ventana principal y navegación entre vistas.
<code>TreeView</code>	Construcción interactiva de función a partir de un árbol.
<code>FunctionView</code>	Construcción de árbol a partir de una función.
<code>CryptoView</code>	Cifrado y descifrado directo usando funciones.
<code>GraphCanvas</code>	Renderizado de grafos y elementos visuales.
<code>GraphLogic</code>	Algoritmos de grafos y construcción de vértebras.
<code>CryptoEngine</code>	Orquestación del cifrado y descifrado Hill.

Tabla 3: Clases principales del sistema.

7.2 Decisiones de diseño

- Se separó la lógica de negocio de la interfaz para facilitar mantenimiento.
- Se reutilizó un único canvas para todas las vistas gráficas.

- Se compartió la generación aleatoria de funciones entre vistas.
- Se mantuvo el estilo visual oscuro y consistente en toda la interfaz.
- El renderizado de loops se integró en el mismo pipeline que las demás aristas para no duplicar lógica.

7.3 Relación entre componentes

La interfaz consulta a **GraphLogic** para construir estructuras y a **CryptoEngine** para transformar texto. **GraphCanvas** solo visualiza el resultado y no contiene lógica de negocio.

8 Capturas y figuras

Las siguientes figuras pueden incluirse posteriormente como capturas del programa en ejecución.



Figura 2: Menú principal de la aplicación.



Figura 3: Vista de construcción de función desde árbol.

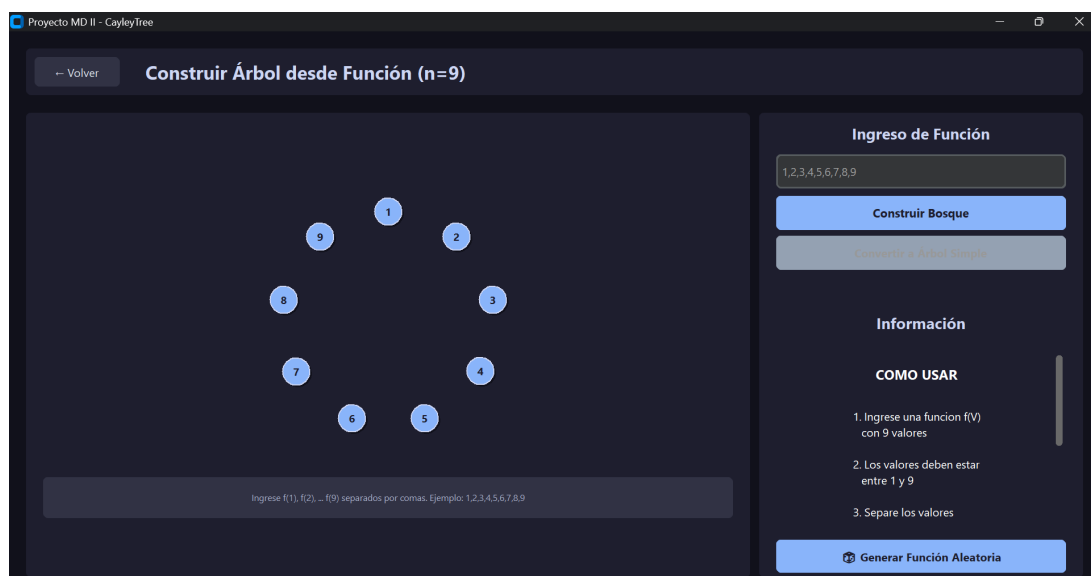


Figura 4: Vista de construcción de árbol desde función.

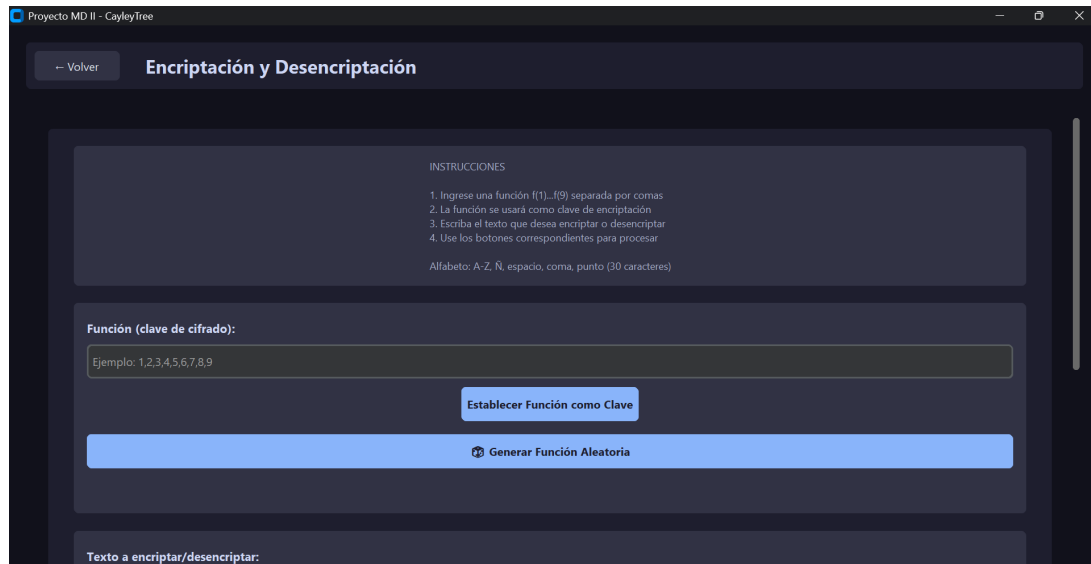


Figura 5: Vista de cifrado y descifrado directo.

9 Posibles mejoras futuras

- Exportar visualizaciones como imagen o PDF.
- Añadir pruebas automatizadas para la lógica de grafos y criptografía.
- Incorporar zoom y desplazamiento en el canvas.
- Mejorar el manejo adaptativo de la geometría de self-loops y aristas densas.
- Añadir una vista comparativa entre la función y su árbol equivalente.
- Registrar historial de funciones y claves generadas.

10 Conclusiones

El repositorio implementa una solución coherente y modular para explorar la relación entre árboles etiquetados y funciones, al mismo tiempo que integra una demostración criptográfica basada en matrices. La separación entre interfaz, lógica de grafos y utilidades matemáticas permite entender, mantener y extender el proyecto de forma razonable. Además, la visualización manual con Canvas brinda control completo sobre la presentación de vértices, aristas, vértebras y bucles.

A Fragmento representativo de código

Listing 3: Fragmento conceptual de generación de matriz desde una función.

```
for i in range(dim):  
    row = []  
    for j in range(dim):  
        row.append((func[i] * (j + 1) + (func[j] + 1)) % MOD)  
    arr.append(row)
```